# INFO8006 Introduction to Artificial Intelligence

Exercise session 6: Machine learning

## Recap

### Problem statement

Let $\mathbf{d} \sim p(\mathbf{x}, y)$ be a dataset of $N$ example input-output pairs

$$\mathbf{d} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_N, y_N)\},$$

where $\mathbf{x}_i \in \mathbb{R}^d$ are $d$-dimensional vectors representing the input values and $y_i \in \mathcal{Y}$ are the corresponding output values. From this data, we want to identify a probabilistic model $p_\theta(y \mid \mathbf{x})$ that best explains the data. The problem is a *regression* problem if $y \in \mathbb{R}$ and a *classification* problem if $y \in \{0, 1, \ldots, C-1\}$.

### Linear regression

Linear regression models $p(y \mid \mathbf{x}) = \mathcal{N}(y \mid \mathbf{w}^T\mathbf{x} + b, \sigma^2)$. Its maximum likelihood estimate minimizes the sum of squared errors $\sum_{i=1}^N (y_i - (\mathbf{w}^T\mathbf{x}_i + b))^2$. Appending a constant feature 1 to $\mathbf{x}$ to absorb $b$ into $\mathbf{w}$, it is $\mathbf{w}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$, where $\mathbf{X}$ stacks the inputs $\mathbf{x}_i^T$ and $\mathbf{y}$ the outputs $y_i$.

### Logistic regression

Logistic regression models $P(Y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x} + b)$, with the sigmoid $\sigma(z) = 1/(1+\exp(-z))$. Its maximum likelihood estimate minimizes the *cross-entropy* $\sum_{i=1}^N -y_i \log \sigma(\mathbf{w}^T\mathbf{x}_i + b) - (1 - y_i)\log(1 - \sigma(\mathbf{w}^T\mathbf{x}_i + b))$, which has no closed-form minimizer.

### Gradient descent

To minimize a loss function $\mathcal{L}(\theta)$, *gradient descent* updates the model parameters iteratively using the update rule

$$\theta_{t+1} = \theta_t - \alpha \nabla_\theta \mathcal{L}(\theta_t),$$

where $\theta_0$ are the initial parameters of the model and $\alpha$ is the *learning rate*. When $\mathcal{L}(\theta) = \frac{1}{N}\sum_i \ell(y_i, f(\mathbf{x}_i; \theta))$, *stochastic gradient descent* replaces the gradient by its average over a random minibatch of the pairs.

### Multilayer perceptron

A *shallow network* with input $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$, $q$ hidden units and one output computes

$$h_j = \sigma\Big(\sum_{i=0}^{d_{\text{in}}-1} w_{ji}x_i + b_j\Big) \quad \text{for } j = 0, \ldots, q-1, \qquad y = \sum_{j=0}^{q-1} v_j h_j + c,$$

where $\sigma$ is an activation function. In matrix form, a hidden layer computes

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}),$$

where $\mathbf{W} \in \mathbb{R}^{q \times d_{\text{in}}}$ is the *weight matrix*, of entries $w_{ji}$, and $\mathbf{b} \in \mathbb{R}^q$ is the *bias vector*. Layers composed in series form a deep network with $K$ hidden layers,

$$\begin{aligned} \mathbf{h}_0 &= \mathbf{x} \\ \mathbf{h}_k &= \sigma(\mathbf{W}_k \mathbf{h}_{k-1} + \mathbf{b}_k) \quad \text{for } k = 1, \ldots, K \\ \mathbf{y} &= \sigma_{\text{out}}(\mathbf{W}_y \mathbf{h}_K + \mathbf{b}_y), \end{aligned}$$

where $\mathbf{W}_k \in \mathbb{R}^{q_k \times q_{k-1}}$ is the weight matrix and $\mathbf{b}_k \in \mathbb{R}^{q_k}$ is the bias vector of the $k$-th layer. The resulting function $f(\mathbf{x}; \theta)$, where $\theta$ denotes the set of all model parameters, is a *multilayer perceptron*. With ReLU units, it is piecewise linear: a shallow network with one input has at most $q + 1$ linear regions, a deep network with $K$ layers of $q$ units up to $(q + 1)^K$.

### Universal approximation theorem

Let $\sigma : \mathbb{R} \to \mathbb{R}$ be continuous and not a polynomial. For every compact set $K \subset \mathbb{R}^{d_{\text{in}}}$, every continuous function $f : K \to \mathbb{R}$ and every $\epsilon > 0$, there exist $q$, $w_{ji}$, $b_j$, $v_j$ and $c$ such that

$$\sup_{\mathbf{x} \in K} \left| f(\mathbf{x}) - \Big( \sum_{j=0}^{q-1} v_j \, \sigma\Big( \sum_{i=0}^{d_{\text{in}}-1} w_{ji} x_i + b_j \Big) + c \Big) \right| < \epsilon :$$

a shallow network with enough hidden units approximates any continuous function on a closed and bounded domain arbitrarily well.

### Activation functions

Activation functions $\sigma : \mathbb{R} \to \mathbb{R}$ introduce non-linearities in the network. Common activation functions include

- the sigmoid $\sigma(x) = \frac{1}{1+\exp(-x)}$, which maps inputs to the range $(0, 1)$;
- the hyperbolic tangent $\sigma(x) = \tanh(x) = \frac{\exp(x)-\exp(-x)}{\exp(x)+\exp(-x)}$, which maps inputs to the range $(-1, 1)$;
- the rectified linear unit $\sigma(x) = \text{ReLU}(x) = \max(0, x)$.

### Output layers

The output layer $\mathbf{y} = \sigma_{\text{out}}(\mathbf{W}_y \mathbf{h}_K + \mathbf{b}_y)$ depends on the task:

- for regression, its width is the dimensionality $d_{\text{out}}$ of the output and $\sigma_{\text{out}}$ is the identity;
- for binary classification, its width is 1 and $\sigma_{\text{out}}$ is the sigmoid, whose output is $P(Y = 1 \mid \mathbf{x})$;
- for classification into $C$ classes, its width is $C$ and $\sigma_{\text{out}}$ is the *softmax*

$$\text{Softmax}(\mathbf{z})_i = \frac{\exp(z_i)}{\sum_{i'=0}^{C-1} \exp(z_{i'})} \quad \text{for } i = 0, \ldots, C-1,$$

  so that $\mathbf{y}$ is a vector of $C$ probabilities $P(Y = i \mid \mathbf{x})$ summing to 1.

### Loss functions

The parameters $\theta$ of a deep network $f(\mathbf{x}; \theta)$ are learned by minimizing a loss function $\mathcal{L}(\theta)$ over the dataset $\mathbf{d} = \{(\mathbf{x}_i, \mathbf{y}_i)\}$ of $N$ input-output pairs. The loss function is derived from the likelihood:

- for regression, assuming a Gaussian likelihood, the loss is the *mean squared error*

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \|\mathbf{y}_i - f(\mathbf{x}_i; \theta)\|^2;$$

- for classification, assuming a categorical likelihood, the loss is the *cross-entropy*, the negative log-probability of the observed class,

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log f_{y_i}(\mathbf{x}_i; \theta).$$

**Chain rule**

The gradient of the loss is obtained by the chain rule. For example, for $h_1 = \sigma(w_1 x + b_1)$, $\hat{y} = w_2 h_1 + b_2$ and $\ell = (y - \hat{y})^2$, the chain $w_1 \to h_1 \to \hat{y} \to \ell$ gives

$$\frac{\partial \ell}{\partial w_1} = \frac{\partial \ell}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_1} \frac{\partial h_1}{\partial w_1} = -2(y - \hat{y})\, w_2\, \sigma'(w_1 x + b_1)\, x.$$

On the *computation graph* of the network (variables and the operations that produce them), the *forward pass* computes every intermediate value from the input to the loss; the *backward pass* then multiplies the local derivatives from the loss back to each parameter, reusing the values of the forward pass. *Automatic differentiation* implements this procedure for any program.

**Generalization**

We want a small loss on new pairs drawn from $p(\mathbf{x}, y)$, not on the pairs of $\mathbf{d}$. An honest estimate comes from a *test set* $\mathbf{d}_{\text{test}}$, held out from training. A model too small for the data *underfits* (both losses stay high); a model large enough to memorize $\mathbf{d}$ *overfits* (the training loss decreases while the test loss increases). *Hyperparameters* (width, depth, learning rate, minibatch size, number of epochs) are chosen on a *validation set*, also held out from training; the test set is used once, at the end.

**Convolution**

For the 1d input $\mathbf{x} \in \mathbb{R}^W$ and the convolutional kernel $\omega \in \mathbb{R}^w$, the discrete *convolution* $\mathbf{x} \circledast \omega$ is a vector of size $W - w + 1$ such that

$$(\mathbf{x} \circledast \omega)[i] = \sum_{m=0}^{w-1} \mathbf{x}_{m+i} \omega_m.$$

Technically, $\circledast$ is the cross-correlation: the kernel is not flipped. Padding the input with $p$ zeros on each side results in an output of size $W - w + 1 + 2p$. For the 3d input tensor $\mathbf{x} \in \mathbb{R}^{C \times H \times W}$ with $C$ channels and the 3d convolutional kernel $\omega \in \mathbb{R}^{C \times h \times w}$, the 2d convolution $\mathbf{x} \circledast \omega$ is a 2d tensor of size $(H - h + 1) \times (W - w + 1)$ such that

$$(\mathbf{x} \circledast \omega)[j, i] = \sum_{c=0}^{C-1} \sum_{n=0}^{h-1} \sum_{m=0}^{w-1} \mathbf{x}_{c,n+j,m+i} \omega_{c,n,m}.$$

**Convolutional neural networks**

A *convolutional layer* is defined by a set of $K$ kernels $\omega_k$ of size $C \times h \times w$. It applies the 2d convolution to the input tensor $\mathbf{x}$ of size $C \times H \times W$ to produce a set of $K$ feature maps $\mathbf{o}_k$, the $K$ channels of its output. With a pooling area of size $h \times w$, *max-pooling* maps a tensor $\mathbf{x} \in \mathbb{R}^{C \times (rh) \times (sw)}$ to a tensor $\mathbf{o} \in \mathbb{R}^{C \times r \times s}$ such that

$$\mathbf{o}_{c,j,i} = \max_{n<h, m<w} \mathbf{x}_{c,hj+n,wi+m}.$$

The *receptive field* of a value is the patch of the input it depends on: a $w \times w$ kernel sees $w \times w$ values, and each further convolutional or pooling layer widens it. A *convolutional neural network* (CNN) uses convolutional layers, pooling layers and fully connected layers.

*In session:* exercises 1, 4, 7.

**Exercise 1 Travel-time heuristic** (Exam, January 2022)
A delivery robot works in a building laid out as a $5 \times 5$ grid of rooms, where each cell $(r, c)$ is a room with doors leading to the adjacent rooms. The robot must reach its charging station as fast as possible, but the station's position is unknown to it. Furthermore, some regions of the building are cluttered, and crossing rooms in these regions takes longer. Fortunately, the other robots of the fleet have logged their best times to the station, starting from a few different rooms. Due to rounding errors, the best times in the log are assumed to be measurements affected by additive Gaussian noise $\mathcal{N}(0, 1)$.

| $i$ | Starting room | Measured best time |
|---|---|---|
| 1 | $(4, 5)$ | 2.0 |
| 2 | $(5, 3)$ | 3.5 |
| 3 | $(3, 3)$ | 4.5 |
| 4 | $(4, 1)$ | 7.0 |
| 5 | $(1, 2)$ | 8.5 |

From the log, the robot's program learns a heuristic approximating the best time to get to the station, starting from room $(r, c)$. It uses a small neural network as approximator, described by the following parametric function,

$$h(r, c; \phi) = \mathrm{ReLU}(rw_1 + cw_2 + w_3) + \mathrm{ReLU}(rw_4 + cw_5 + w_6)$$
$$\mathrm{ReLU}(z) = \max(z, 0),$$

where $\phi = (w_1, w_2, w_3, w_4, w_5, w_6)$ is the set of parameters/weights of the neural network.

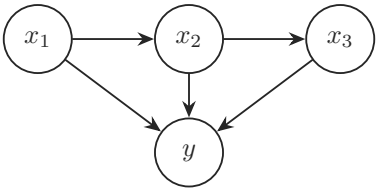
(a) Among the following sets of parameters ($A$, $B$ or $C$), which one would you use? Justify your answer.

| Set | $w_1$ | $w_2$ | $w_3$ | $w_4$ | $w_5$ | $w_6$ |
|---|---|---|---|---|---|---|
| $\phi_A$ | $-1.5$ | 1 | 4 | 1 | $-1.5$ | 6 |
| $\phi_B$ | $-1$ | 1.5 | 3 | 0 | $-1$ | 4 |
| $\phi_C$ | $-2$ | 0.5 | 4.5 | 1.5 | 0 | 5 |

(b) You now assume a Gaussian prior $\mathcal{N}(0, 1)$ on each parameter. Which set of parameters in the table above would you now choose? Justify your answer.

(c) Discuss the procedure you would implement on a computer to find the optimal set of parameters, had the table above not been provided.

**Exercise 2 Predict the leaderboard score**
A leaderboard ranks language models by an overall score. The Bayesian network below represents how this score is computed. In this model, $x_1$, $x_2$ and $x_3$ respectively denote the scores obtained by a model on a validation set, on a public benchmark and on a held-out test set. The three evaluations share the same harness, and a model that does well on one tends to do well on the next. In particular, $x_2 \sim \mathcal{N}(a_1 x_1 + \mu_2, \sigma_2^2)$ and $x_3 \sim \mathcal{N}(a_2 x_2 + \mu_3, \sigma_3^2)$. Finally, $y \sim \mathcal{N}(a_3 x_1 + a_4 x_2 + a_5 x_3 + \mu_y, \sigma_y^2)$ stands for the overall score, which is a linear combination of the three scores plus some Gaussian noise due to rounding errors. Answer the following questions about this model.




(a) Assuming the parameters of the model are known, what is the expected value of $y$ given $x_1$ and $x_2$?

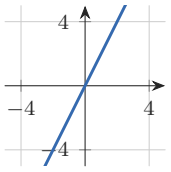
(b) Suppose now that the parameters are unknown, and that the scores $x_3$ on the held-out test set are not observed. Given a learning set $\mathbf{d} = \{(x_{i,1}, x_{i,2}, y_i)\}$ of $N$ independent and identically distributed points, derive the maximum likelihood estimate of the parameters of $p(y \mid x_1, x_2)$.

## Exercise 3 Heteroscedastic linear regression

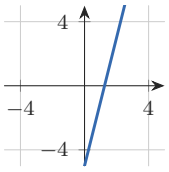
In linear regression with Gaussian noise of constant variance, $y_i \sim \mathcal{N}(w^T x_i, \sigma^2)$, the maximum likelihood estimate of the weights is $w_{\text{MLE}} = (X^T X)^{-1} X^T Y$, where the rows of $X$ are the inputs $x_i^T$ and $Y = (y_i)$. What does this estimate become if the noise is different for each sample, *i.e.* $y_i \sim \mathcal{N}(w^T x_i, \sigma_i^2)$ with known variances $\sigma_i^2$?

## Exercise 4 Neural network representations (CS188, Spring 2024)

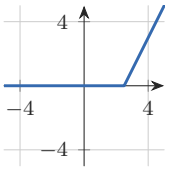
You are given a number of functions (a-h) of a single variable, $x$, which are graphed below. For each network structure proposed afterwards, indicate which functions they are able to represent. When possible, indicate appropriate values for the parameters.



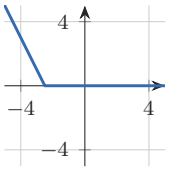

(a) $2x$



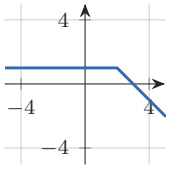

(b) $4x - 5$



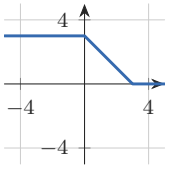

(c) $\begin{cases} 2x - 5 & x \geq 2.5 \\ 0 & x < 2.5 \end{cases}$



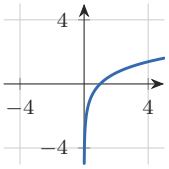

(d) $\begin{cases} -2x - 5 & x \leq -2.5 \\ 0 & x > -2.5 \end{cases}$



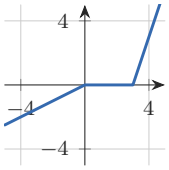

(e) $\begin{cases} -x + 3 & x \geq 2 \\ 1 & x < 2 \end{cases}$




(f) $\begin{cases} 3 & x \leq 0 \\ 3 - x & 0 < x \leq 3 \\ 0 & x > 3 \end{cases}$

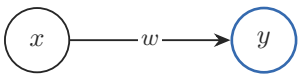



(g) $\log(x)$

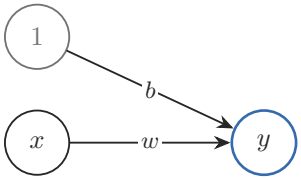



(h) $\begin{cases} 0.5x & x \leq 0 \\ 0 & 0 < x \leq 3 \\ 3x - 9 & x > 3 \end{cases}$

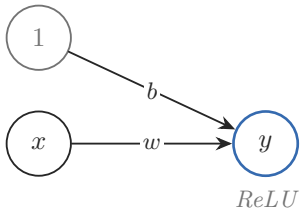
(a) We start with a linear transformation of the scalar input $x$, weight $w$, and output $y$, such that $y = wx$.



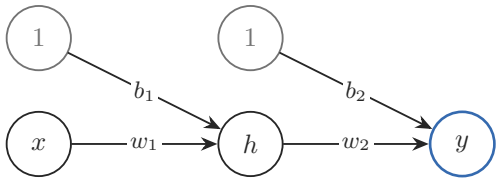

(b) We then introduce a bias term $b$, such that $y = wx + b$.



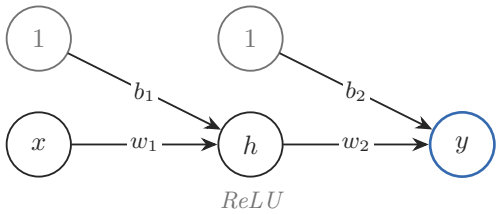
(c) We now introduce a non-linearity into the network, the ReLU $\text{ReLU}(x) = \max(0, x)$, so that $y = \text{ReLU}(wx + b)$.



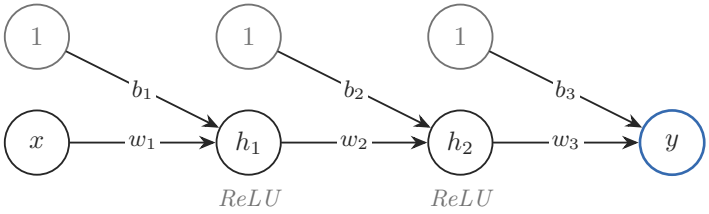

(d) Now we consider neural networks with multiple affine transformations, $h = w_1 x + b_1$ and $y = w_2 h + b_2$, as depicted below.



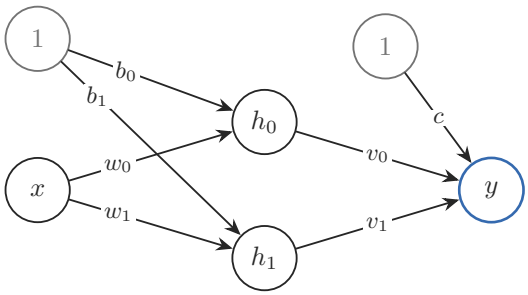

(e) We now add a ReLU non-linearity to the network between the affine transformations: $h = \text{ReLU}(w_1 x + b_1)$ and $y = w_2 h + b_2$.



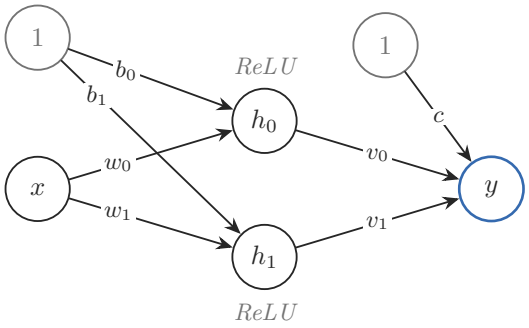

(f) Now we add another hidden layer to the network, as depicted below: $h_1 = \text{ReLU}(w_1 x + b_1)$, $h_2 = \text{ReLU}(w_2 h_1 + b_2)$ and $y = w_3 h_2 + b_3$. You do not have to guess parameters values anymore.




(g) We'd like to consider using a neural net with just one hidden layer, but larger. Let's first consider using just two affine functions, with no nonlinearity in between, $h_j = w_j x + b_j$ for $j = 0, 1$ and $y = v_0 h_0 + v_1 h_1 + c$. You do not have to guess parameters values anymore.



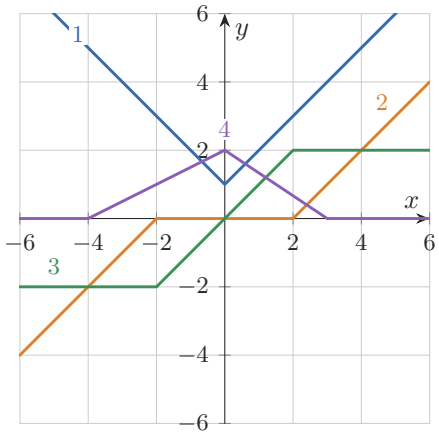
(h) Now we'll add a non-linearity between the two affine layers, $h_j = \text{ReLU}(w_j x + b_j)$ for $j = 0, 1$ and $y = v_0 h_0 + v_1 h_1 + c$, to produce the neural network below with a hidden layer of size 2. You do not have to guess parameters values anymore.




(i) Are there functions that cannot be represented by any of the proposed networks? If yes, explain why and what you would need to model them.

## Exercise 5 Piecewise-linear functions

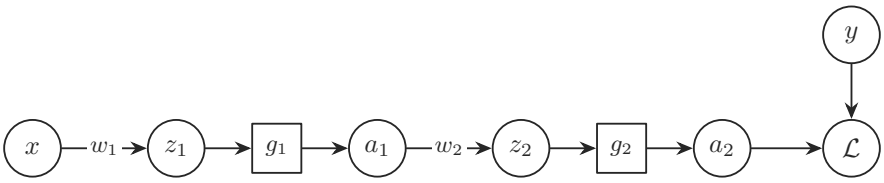
For each of the piecewise-linear functions below, write a function $y = f(x)$ as a composition of sums $(+, -)$, ReLU ($\text{ReLU}(x) = \max(x, 0)$) non-linearities, and real parameters (weights and biases) that exactly matches the function over $\mathbb{R}$.




For example, $y = \text{ReLU}(x + 2) - \text{ReLU}(-2x)$ is a valid function.

## Exercise 6 Binary classification (CS188, Spring 2024)

Consider the following simple neural network for binary classification. Here $x$ is a single real-valued input feature with an associated class $y$ (0 or 1). There are two weight parameters $w_1$ and $w_2$, and non-linearity functions $g_1$ and $g_2$. The network will output a value $a_2$ between 0 and 1, representing the probability of being in class 1. We will be using a loss function $\mathcal{L}$ to compare the prediction $a_2$ with the true class $y$.




(a) Write down the forward pass on this network, writing the output values for each node $z_1$, $a_1$, $z_2$ and $a_2$ in terms of the node's input values.

(b) Derive the arguments of the loss $\mathcal{L}(a_2, y)$ in terms of the input $x$, weights $w_i$, and activation functions $g_i$.

(c) Using the chain rule, differentiate $\mathcal{L}$ with respect to $w_2$. Write your expression as a product of partial derivatives at each node: *i.e.* the partial derivative of the node's output with respect to its inputs. (Hint: the series of expressions you wrote in (a) will be helpful; you may use any of those variables.)

(d) Motivate a choice for the output activation function $g_2$ given the nature of the problem. Motivate also a loss function. Give the expression of the gradients of both functions with respect to their inputs.

(e) We set the loss function to the binary cross-entropy

$$\mathcal{L}(a_2, y) = -y \log a_2 - (1 - y) \log(1 - a_2),$$

and $g_1$ and $g_2$ are both sigmoid functions $\sigma(z) = \frac{1}{1+e^{-z}}$. If you made this choice in the previous question, use your results to get the expression of $\frac{\partial \mathcal{L}}{\partial w_2}$. Otherwise, start by expressing $\frac{\partial \sigma(z)}{\partial z}$ and $\frac{\partial \mathcal{L}}{\partial a_2}$.

(f) Now use the chain rule to express $\frac{\partial \mathcal{L}}{\partial w_1}$ as a product of partial derivatives at each node of interest.

(g) Finally, write $\dfrac{\partial \mathcal{L}}{\partial w_1}$ in terms of $x$, $y$, $w_i$, $a_i$, $z_i$.

(h) What is the gradient descent update for $w_1$ with learning rate $\alpha$ in terms of the values computed above?

**Exercise 7 Training loop** (Exam, January 2024)

Let us consider a neural network $f$ with one hidden layer taking as input a scalar $x \in \mathbb{R}$ and producing as output a prediction $\hat{y} = f(x; \theta) \in \mathbb{R}$. The neural network is defined as

$$f(x; \theta) = w_1 \text{ReLU}(w_2 x + w_3) + w_4 \text{ReLU}(w_5 x + w_6) + w_7 \text{ReLU}(w_8 x + w_9),$$

where $\theta = (w_1, w_2, w_3, w_4, w_5, w_6, w_7, w_8, w_9)$ is the set of parameters and $\text{ReLU}(x) = \max(x, 0)$ is the rectified linear unit function.

(a) Draw the computation graph representing the neural network and the flow of information from inputs to outputs. Your diagram should be a directed graph that follows the following conventions:

- circled nodes correspond to variables (input, output, parameters or intermediate variables),
- squared nodes correspond to primitive operations (addition, multiplication, ReLU) and produce an intermediate variable as output,
- directed edges correspond to the flow of information, from inputs to outputs.

(b) For $\theta = (-1, \frac{1}{2}, 0, -4, 1, -2, 4, 1, -5)$, draw the function $\hat{y} = f(x; \theta)$ for $x \in [-10, 10]$.

(c) Using the data point $(x, y) = (10, -15)$ and the value of $\theta$ given above, we want to fine-tune the parameter $w_8$ of the neural network such that $f(x; \theta)$ produces a more accurate prediction of the target $y$.

i. Evaluate the squared error loss at the data point $(x, y) = (10, -15)$ and the current value of $\theta$.

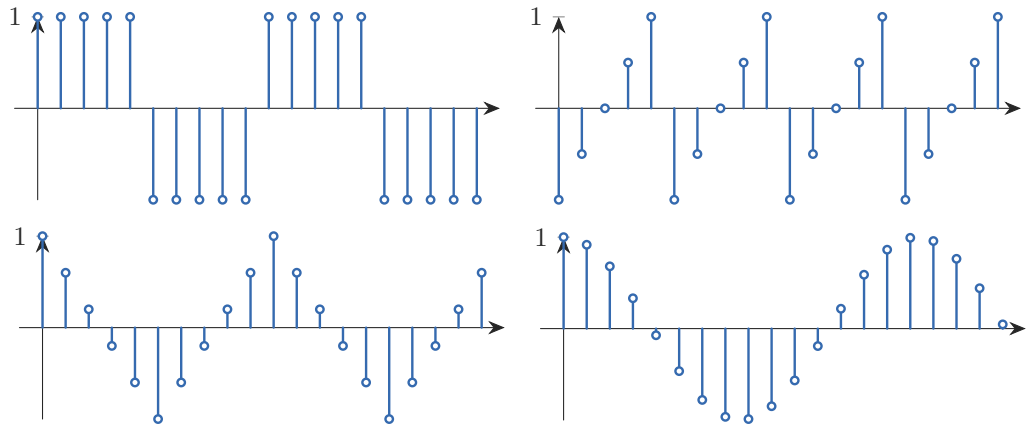
ii. Derive an expression for the derivative of the squared error loss with respect to $w_8$.

iii. Update the parameter $w_8$ using one step of gradient descent with a learning rate $\alpha = 0.0005$.

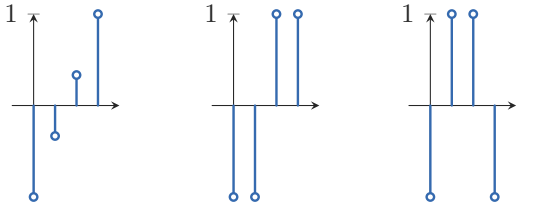
iv. Verify if the value of the loss function has decreased after the update.

## Exercise 8 1d convolution

Consider the following four signals $x \in \mathbb{R}^{20}$, numbered 1 to 4 from left to right and top to bottom,




and the following three convolution kernels $\omega \in \mathbb{R}^4$.




$(-1, -\frac{1}{3}, \frac{1}{3}, 1)$ $(-1, -1, 1, 1)$ $(-1, 1, 1, -1)$

(a) Compute the convolution $x \circledast \omega$ of the first signal (the square wave) with each kernel, without padding, so that each output has $20 - 4 + 1 = 17$ values.

(b) Without computing every value, sketch the convolution of the three other signals with each kernel. What does each kernel detect?

## Exercise 9 2d convolution

(a) Represent the 2D convolution $x \circledast \omega$ of the following greyscale image $x$, where a value of 0 corresponds to a white pixel and a value of 1 corresponds to a black pixel, with the following convolution kernel $\omega$ (the box marks the window of the first output value).

$x \in \mathbb{R}^{5\times5}$

| | | | | |
|---|---|---|---|---|
| 0.1 | 0.3 | 0.6 | 0.9 | 0.4 |
| 0.2 | 0.5 | 0.8 | 0.7 | 0.3 |
| 0.4 | 0.7 | 1.0 | 0.8 | 0.5 |
| 0.2 | 0.6 | 0.9 | 0.7 | 0.3 |
| 0.1 | 0.4 | 0.7 | 0.5 | 0.2 |

$\omega \in \mathbb{R}^{2\times3}$

| | | |
|---|---|---|
| 0.2 | 0.6 | 0.9 |
| 0.4 | 0.8 | 0.3 |

Is the result dependent on the choice of coordinate system (*i.e.* on which directions are considered the "$i$" and "$j$" directions in $x[j, i]$ and $\omega[j, i]$)?

(b) The image $x$ is now a colour image, $x \in \mathbb{R}^{3\times4\times4}$, with three channels (one for the red colour, one for the green colour and one for the blue colour):

$x[0,:,:]$ (red)

| | | | |
|---|---|---|---|
| 0.1 | 0.3 | 0.6 | 0.9 |
| 0.2 | 0.5 | 0.8 | 0.7 |
| 0.4 | 0.7 | 1.0 | 0.8 |
| 0.2 | 0.6 | 0.9 | 0.7 |

$x[1,:,:]$ (green)

| | | | |
|---|---|---|---|
| 0.2 | 0.2 | 0.2 | 1.0 |
| 0.2 | 0.2 | 0.2 | 1.0 |
| 0.2 | 0.2 | 0.2 | 1.0 |
| 0.2 | 0.2 | 0.2 | 1.0 |

$x[2,:,:]$ (blue)

| | | | |
|---|---|---|---|
| 0.2 | 0.2 | 0.2 | 0.1 |
| 0.2 | 0.2 | 0.2 | 0.1 |
| 0.2 | 0.2 | 0.2 | 0.1 |
| 0.2 | 0.2 | 0.2 | 0.1 |

Represent the 2D convolution $x \circledast \omega$ with the following convolution kernel $\omega \in \mathbb{R}^{3\times2\times3}$:

$\omega[0,:,:]$

| | | |
|---|---|---|
| 0.2 | 0.6 | 0.9 |
| 0.4 | 0.8 | 0.3 |

$\omega[1,:,:]$

| | | |
|---|---|---|
| 0.1 | 0.1 | 0.1 |
| 0.1 | 0.1 | 0.1 |

$\omega[2,:,:]$

| | | |
|---|---|---|
| 0.2 | 0.2 | 0.2 |
| 0.2 | 0.2 | 0.2 |

Is the result dependent on the choice of coordinate system? What happens if the channels of $x$ are shuffled?

### Exercise 10 Problem statement, output layer and loss function

A dataset $\mathbf{d}$ consists of $N = 500$ images of cats and dogs, with exactly one cat or one dog appearing in each image. Each image has a spatial resolution of $32 \times 32$ pixels and is encoded in RGB format using three channels of real-valued entries. The first channel represents the red component of the image, the second channel represents the green component, and the third channel represents the blue component.

You would like to formulate a supervised learning problem in order to develop a deep learning model suited to a given task.

You are interested in three different tasks:

1. Assign a beauty score (a real-valued quantity) to a given image.
2. Determine whether the image contains a cat or a dog.
3. Determine whether each pixel of an image belongs to a cat, a dog or the background.

For each problem, you want to identify a probabilistic model in the form $p_\theta(y \mid x)$ from a dataset that you will construct:

$$\mathbf{d} = \{(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N)\}.$$

(a) For each task, define the random variables $X$ and $Y$, specify their respective dimensions, and explain what $\theta$ represents.

Once the task has been formulated, you construct the dataset $\mathbf{d}$ by annotating each of the 500 images, thereby obtaining a sample $y_i \sim Y$ associated with a sampled image $x_i \sim X$.

(b) Show that finding the parameters $\theta$ that maximize the likelihood of your model over a sample of $\mathbf{d}$,

$$p\left((x_i, y_i) \mid \theta\right),$$

is equivalent to finding the parameters $\theta$ that maximize the probability of the modelled conditional distribution over this sample,

$$p\left(y_i \mid x_i, \theta\right).$$

After reviewing the literature, you choose a model for $p_\theta(y \mid x) = p(y \mid x, \theta)$.

For the first task, you assume that a suitable model is to model $Y \mid X$ as a Gaussian distribution with homoscedastic variance. For the second task, a suitable model is a Bernoulli distribution. For the third task, a suitable model is a categorical distribution.

(c) For each task, propose a suitable loss function to train your neural network. Propose also an output layer for your neural network in the form

$$\mathbf{y} = \sigma_{\text{out}}\left(\mathbf{W}_y \mathbf{h} + \mathbf{b}_y\right).$$

Let $\mathbf{h}$ denote the output of the layer immediately preceding the output layer, with dimension $d_o$ ($\mathbf{h} \in \mathbb{R}^{d_o}$). Specify the dimensions of $\mathbf{y}$, $\mathbf{W}_y$, and $\mathbf{b}_y$ and give an example of a suitable activation function $\sigma_{\text{out}}$ for each task.

*Hint:* For each task, which parameter of the distribution $p_\theta(y \mid x)$ is estimated by your neural network?

You want to solve a new task: estimate the age of the dog or the cat in the image. To simplify the problem, you consider the age as an unrestricted real-valued quantity. After some reflection, it becomes clear that directly inferring the age from the image is not a good idea. If the type of animal (cat or dog) were first estimated, it would be much easier to estimate the animal's age from the image and this information.

(d) Formulate the task in the context of supervised learning and explain all the elements appearing in $p_\theta(y \mid x)$. What mathematical quantity is estimated by the neural network?

(e) Draw a Bayesian network modelling the image, the type of animal and its age. Find a new expression for $p(y \mid x)$ that makes the effect of the type of the animal in the image explicit.

(f) Consider a large neural network designed to solve the task, divided into multiple submodules, each implemented as a separate neural network. Use the image classification model developed above as one of these submodules. Define the inputs and outputs of the additional submodules required to solve the task. Relate the resulting architecture to the mathematical expression derived above. Which submodule estimates each of the distributions appearing in this expression?

*Hint:* Derive an expression for $\mathbb{E}\{Y \mid X\}$ using the Bayesian network, explicitly accounting for the animal type $A$.

**Exercise 11 Shallow network expressiveness** (Exam, August 2026, adapted)

We consider a neural network for binary classification that takes as input a 2D point $(x_1, x_2) \in \mathbb{R}^2$ and outputs a probability $\hat{y} = f(x_1, x_2) \in [0, 1]$. The network has one hidden layer with two units and is defined as

$$\begin{aligned} h_0 &= \text{ReLU}(x_1 - 1), \\ h_1 &= \text{ReLU}(x_2 - 2), \\ \hat{y} &= \sigma(h_0 + h_1 - 2), \end{aligned}$$

where

$$\text{ReLU}(z) = \max(0, z)$$

is the rectified linear unit and

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

is the sigmoid function.

The predicted class is

$$\hat{c} = \begin{cases} 1, & \text{if } \hat{y} \geq 0.5, \\ 0, & \text{otherwise.} \end{cases}$$

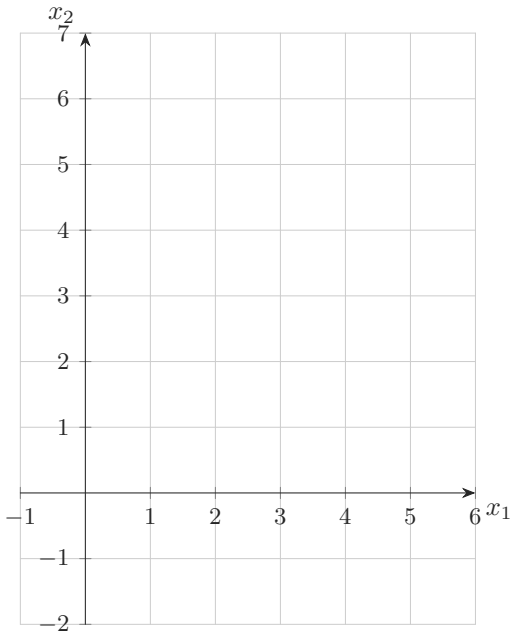
(a) Draw the computation graph representing the neural network. Use circled nodes for variables (inputs, outputs, and intermediate values) and squared nodes for operations (addition, ReLU, and sigmoid).

(b) The ReLU activation functions divide the input space into several regions, depending on whether each hidden unit is active ($h_j > 0$) or inactive ($h_j = 0$).

i. Identify and characterize these regions in terms of conditions on $x_1$ and $x_2$.

ii. For each region, give a simplified expression for the output probability $\hat{y}$ as a function of $x_1$ and $x_2$.

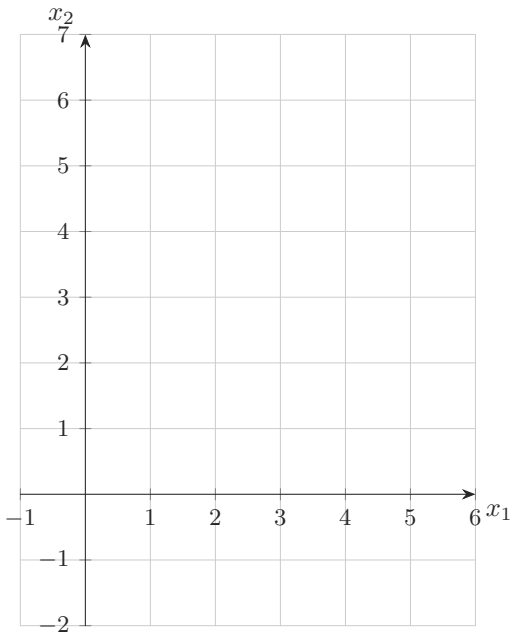
(c) Plot the decision boundary $\hat{y} = 0.5$ of the classifier on the grid below. Clearly indicate which region(s) are classified as class 1 and which are classified as class 0.




(d) We now consider a modified network with a third hidden unit:

$$
\begin{aligned}
h_0 &= \mathrm{ReLU}(x_1 - 1), \\
h_1 &= \mathrm{ReLU}(x_2 - 2), \\
h_2 &= \mathrm{ReLU}(-2x_1 - x_2 + 5), \\
\hat{y} &= \sigma(h_0 + h_1 + h_2 - 2).
\end{aligned}
$$

On the grid below, plot the new decision boundary and compare it to the original one. What is the effect of adding $h_2$?




**Exercise 12 Neural network parameters**

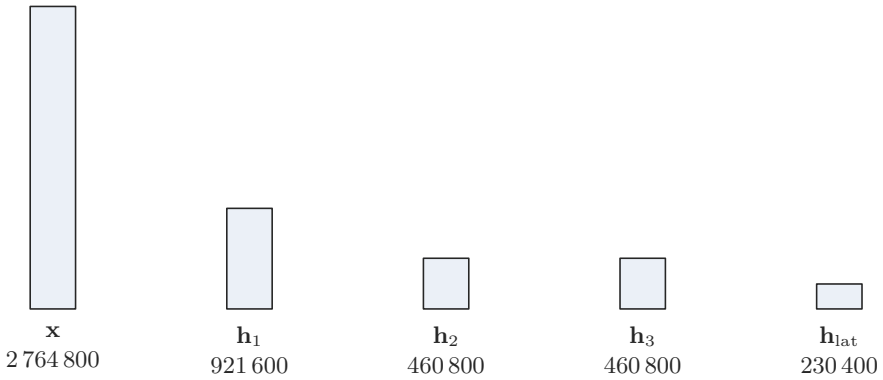
Two teams, A and B, each design a neural network for a challenging task: compressing a 720p HD color image ($720 \times 1280$ pixels). The image is represented using three channels corresponding to the red, green, and blue components, $\mathbf{x} \in \mathbb{R}^{3\times720\times1280}$.

Both teams use an encoder network and a decoder network. The first network, the *encoder*, transforms the input image into a low-dimensional latent representation $\mathbf{h}_{\text{lat}}$, which constitutes the compressed representation. The second network, the *decoder*, reconstructs the original image as faithfully as possible from this low-dimensional representation. We define the *compression ratio* $\eta$ of an encoder as the number of values of the image divided by the number of values of its latent representation.

Team A uses a multilayer perceptron with ReLU activations for its encoder. It flattens the image into a single vector before feeding it to the network, and each block $k$ of its encoder consists of a linear projection followed by the activation:

$$\mathbf{h}_k = \sigma\left(\mathbf{W}_k\mathbf{h}_{k-1} + \mathbf{b}_k\right), \quad \mathbf{h}_0 = \mathbf{x}.$$

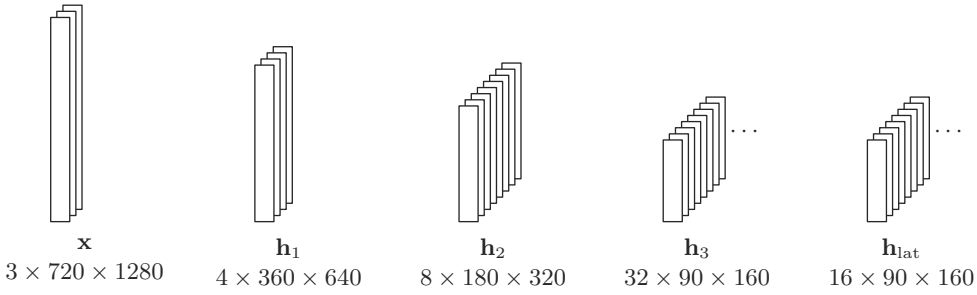
The dimensions of the different layers of team A's encoder are given below:




(a) How many trainable parameters does team A's encoder contain? Is this network realistic to implement on a standard laptop? Assume that each parameter is encoded using 32 bits.

(b) What is the compression ratio achieved by team A's encoder?

Team B uses a convolutional neural network, also with ReLU activations, for its encoder.
Each block of the network consists of a convolutional layer with $3 \times 3$ kernels, followed by a ReLU activation and, in every block but the last, a max-pooling layer with a pooling area of $2 \times 2$. Before being passed through each block, the input feature maps of each block are padded with a one-pixel-wide border of zeros. The shapes of the different layers of team B's encoder are given below, as channels $\times$ height $\times$ width:




(c) How many parameters does team B's encoder contain? Is this network realistic to implement on a standard laptop? Assume that each parameter is encoded using 32 bits.

(d) What is the compression ratio achieved by team B's encoder?

(e) On how many pixels of the image does one value of the latent representation depend, for each team?

**Exercise 13 Learning to play Pacman** (Exam, August 2020)
You observe a Grandmaster agent playing Pacman. How can you use the moves you observe to train your own agent?

(a) Describe formally the data you would collect, the inference problem you would consider, and how you would solve it.

(b) How would you design a neural network to control your agent? Define mathematically the neural network architecture, its inputs, its outputs, its parameters, as well as the loss you would use to train it.

(c) Discuss the expected performance of the resulting agent when 1. the Grandmaster agent is optimal, and 2. the Grandmaster agent is suboptimal.

## Quiz

1. We build a MLP with 2 hidden layers of 32 units, ReLU activations and squared error loss. We discard the biases from our linear layers. We know that our input is a vector of 2 elements and the output is a scalar. How many parameters do we have in the network?
   - ◯ 1024.
   - ◯ 1120.
   - ◯ 1185.
   - ◯ Another value than the ones proposed above.
2. We add biases in the network proposed above. What is the number of parameters of the network?
   - ◯ Another value than the ones proposed below.
   - ◯ 1089.
   - ◯ 1185.
   - ◯ 2048.
   - ◯ 2240.

3. We have an image of shape $(H, W)$, we want to pass it through a layer which reduces each dimension by a constant $K$, *i.e.* produces an output image of shape $(H - K, W - K)$.
   - ◯ It can be done with a convolution layer with a kernel $(K, K)$ or a linear layer with weight matrix $(HW, (H - K)(W - K))$.
   - ◯ It can be done with a convolution layer with a kernel $(K + 1, K + 1)$ or a linear layer with weight matrix $(HW, (H - K)(W - K))$.
   - ◯ It can be done with a convolution layer with a kernel $(K - 1, K - 1)$ or a linear layer with weight matrix $(HW, KK)$.
   - ◯ It can be done with a convolution layer with a kernel $(K + 1, K + 1)$ or a linear layer with weight matrix $(HW, KK)$.
4. From the previous question, we double the number of input pixels by setting the shape of the input image to $(H', W)$ with $H' = 2H$. The latter implies that the output image should be of shape $(H' - K, W - K)$. If we only look at weight parameters (no biases)...
   - ◯ The number of parameters of both the linear and convolution layer are doubled.
   - ◯ The linear layer will have twice more parameters and the convolution twice less.
   - ◯ The linear layer will have at least four times more parameters and the convolution will not change.
   - ◯ Nothing changes neither for the linear layer nor the convolution one.
5. In deep learning, a layer in a multi-layer perceptron is defined as ...
   - ◯ $\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$, where $\sigma$ is the standard deviation function.
   - ◯ $\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$, where $\sigma$ is an activation function, such as the sigmoid function.
   - ◯ $\mathbf{h} = \sigma(\mathbf{W} + \mathbf{x} - \mathbf{b})$, where $\mathbf{W} \in \mathbb{R}^{q \times d}$ is matrix of weights.
   - ◯ $\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$, where $\mathbf{b} \in \mathbb{R}^d$ is the most likely vector of hidden states given $\mathbf{x}$.
6. Arnaud is trying to perform gradient descent on a function $f(x)$ using the update

$$x_{t+1} := x_t - \frac{\partial f}{\partial x}(x_t).$$

   Is gradient descent guaranteed to converge to the global minimum of $f$?
   - ◯ Yes, since he's updating using the gradient of $x$.
   - ◯ Yes, but not for the reason above.
   - ◯ No, since he is updating $x$ in the wrong direction.
   - ◯ No, but not for the reason above.